StudentOS Nexus — Complete Project Architecture & Team Execution Plan

Project Title:
StudentOS Nexus: Intelligent Academic, Career & Cognitive Operating System for Students

This document contains: Complete project vision Modules structure AI architecture overview RAG pipeline Tech stack System engineering goals Team workload division Weekly roadmap Daily execution plans

Core Objective

```
Build a startup-level AI-powered ecosystem that helps students:
- learn smarter
- prepare for placements
- revise efficiently
- improve coding skills
- track productivity
- reduce burnout
- organize academic resources intelligently

The platform continuously learns from:
- uploaded notes
- DSA activity
- study behavior
- interview performance
- productivity patterns
- user interactions
```

Complete Modules

```
MODULE 1 — Authentication & User Management
- Signup/Login
- JWT Authentication
- Google OAuth
- User profiles

MODULE 2 — Dashboard & Analytics Hub
- Study stats
- Productivity analytics
- Revision reminders
- Placement readiness

MODULE 3 — DSA Intelligence Engine
- LeetCode integration
- Weak-topic detection
- Smart recommendations

MODULE 4 — AI Notes Engine (CORE MODULE)
- PDF/PPT/DOC upload
- Semantic search
- AI summaries
- Quiz generation
- Flashcards
- Personalized retrieval

MODULE 5 — AI Mock Interviewer
- HR interviews
- Technical interviews
- Resume-aware questioning
- AI evaluation

MODULE 6 — Resume Analyzer & Career Mentor
- ATS scoring
- Skill gap analysis
- Career roadmap

MODULE 7 — Study Planner & Revision Intelligence
```

```
- Adaptive timetable
- Smart revision scheduling
- Goal tracking

MODULE 8 — Productivity & Cognitive Analytics
- Focus score
- Burnout prediction
- Productivity heatmaps

MODULE 9 — AI Recommendation Engine
- DSA recommendations
- Revision timing
- Skill roadmap

MODULE 10 — AI Memory System
- conversational memory
- semantic memory
- behavioral memory

MODULE 11 — Notifications & Smart Reminders
- revision reminders
- burnout alerts
- streak tracking

MODULE 12 — Admin & Analytics Panel
- user analytics
- AI monitoring
- system health tracking
```

RAG Pipeline

```
PDF Upload
↓
Text Extraction
↓
Cleaning & Preprocessing
↓
Chunking with Overlap
↓
Embedding Generation
↓
Vector Database Storage
↓
Metadata Indexing
↓
Semantic Retrieval
↓
Reranking
↓
LLM Context Injection
↓
LLM Response Generation
```

Tech Stack

```
Frontend:
- React
- Tailwind CSS
- Chart.js/Recharts

Backend:
- FastAPI

Database:
- PostgreSQL

Vector Database:
- ChromaDB

AI Framework:
- LangChain

Embeddings:
```

```
- Sentence Transformers

LLM:
- Gemini API

ML:
- Scikit-learn

Storage:
- Supabase Storage

Deployment:
- Vercel + Render
```

Architecture Principles

```
The architecture must:
- be modular
- support scalability
- support personalization
- follow production-grade engineering practices
- use RAG correctly
- separate AI pipelines cleanly
- support async processing
- support future AI expansion

Database Responsibilities:
- PostgreSQL → structured data
- ChromaDB → embeddings + memory
- Supabase Storage → PDFs/files
- Gemini API → reasoning and generation
```

Team Division

```
PERSON A — AI Systems & Intelligence Engineer

Responsibilities:
- RAG pipelines
- embeddings
- ChromaDB
- recommendation engine
- ML models
- AI memory system
- interview AI
- semantic retrieval
- AI APIs

PERSON B — Platform & Product Engineer

Responsibilities:
- frontend architecture
- React dashboards
- authentication
- PostgreSQL
- planner system
- notifications
- admin panel
- deployment
- DevOps
```

Remote Collaboration Strategy

```
Rules:
1. Define API contracts before coding
2. Use Git feature branches
3. Freeze DB schema early
4. Weekly integration meetings
5. Shared types/interfaces
6. Use one shared frontend and backend
Git Workflow:
```

```
main
develop
feature/rag
feature/dashboard
feature/auth
feature/interviewer
```

Execution Roadmap

```
Week 1:
- architecture
- Git setup
- DB schema
- FastAPI + React setup

Week 2:
- Authentication
- Core RAG system

Week 3:
- AI Notes Engine
- Dashboard UI

Week 4:
- DSA Analytics
- Productivity tracking

Week 5:
- AI Interviewer
- Memory System

Week 6:
- Recommendations
- Study Planner

Week 7:
- Full integration
- Deployment
- Optimization

Week 8:
- Final polish
- Documentation
- Presentation preparation
```

Daily Execution Plan

Week	Person A (AI Engineer)	Person B (Platform Engineer)
1	RAG architecture, ChromaDB, embeddings	React setup, DB schema, auth planning
2	PDF extraction, chunking, retrieval	JWT auth, Google OAuth, dashboard layout
3	Gemini integration, quizzes, flashcards	Notes UI, charts, analytics
4	LeetCode analytics, recommendations	DSA dashboard, productivity UI
5	Interview AI, memory system	Interview UI, results dashboard
6	Recommendation engine, ML models	Planner UI, reminders, notifications
7	Optimization, caching, deployment	Responsive fixes, deployment, admin panel
8	AI tuning, documentation	UI polish, presentation prep

Final Goal: Build a production-style AI-powered student operating system that demonstrates: - AI engineering - RAG architecture - personalization - modern full-stack development - scalable system design - startup-level product thinking The strongest differentiator of the project is: Behavior Analysis + Memory + RAG + Personalization + Cognitive Intelligence.